Contents

Merge Intervals — Complete Interview Handbook

Pattern #8 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Arrays / Intervals section

---

Table of Contents

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Merge Intervals handles problems involving a **collection of ranges** `[start, end]`, typically requiring you to detect overlaps, merge overlapping ranges into consolidated ones, insert a new range, or find gaps/intersections between ranges — almost always after **sorting by start (or end) time**.

1.2 Why Was This Pattern Invented?

Comparing every pair of intervals for overlap naively costs $O(n^2)$. But once intervals are **sorted by start time**, any overlap must occur between **adjacent** intervals in that sorted order (if interval A doesn't overlap with the interval immediately after it, it can't overlap with anything further away either, since all later intervals start even later). This sorted-adjacency property collapses the problem to a single O(n log n) pass (dominated by the sort).

1.3 Real Intuition Behind The Pattern

Imagine calendar meetings scattered throughout the day. If you line them up by start time, two meetings can only possibly overlap if they're **next to each other** in that lineup — a meeting starting at 9am can't overlap with one starting at 5pm if there's a meeting starting at noon that already doesn't overlap with the 9am one (since the noon one starts after the 9am one ends, and 5pm is even later). Sorting converts a global overlap-checking problem into a local, adjacent-pair-checking problem.

1.4 Mental Model

Maintain a "current merged interval." Walk through the sorted list; if the next interval's start is `<= current merged interval's end`, absorb it (extend the end if needed); otherwise, the current merged interval is finalized — start a new one.

1.5 Visual Explanation

```
Intervals (unsorted): [8,10], [1,3], [2,6], [15,18]
Sorted by start:      [1,3], [2,6], [8,10], [15,18]

current = [1,3]
next=[2,6]: 2 <= 3 → overlap → merge → current = [1,6]
next=[8,10]: 8 > 6 → no overlap → finalize [1,6], current = [8,10]
next=[15,18]: 15 > 10 → no overlap → finalize [8,10], current = [15,18]
Finalize [15,18]

Result: [[1,6], [8,10], [15,18]]
```

1.6 Simple Analogy

Merge Intervals is like **consolidating overlapping reservation blocks on a hotel room's booking calendar** — you line up bookings by check-in date, and any two bookings that overlap get merged into one continuous "occupied" block, sweeping left to right exactly once.

1.7 When Should I Immediately Think About Using This Pattern?

- Problem mentions **"intervals," "meetings," "ranges," "schedule."**
- You need to **merge overlapping ranges**, **insert a new range**, or **detect conflicts**.
- You need the **minimum number of resources** (e.g., meeting rooms) to handle overlapping ranges.

- You need to find **free/gaps** between busy intervals.

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“merge overlapping intervals”	Direct signal
“insert interval”	Merge Intervals variant
“meeting rooms,” “conference rooms”	Interval overlap counting
“non-overlapping intervals”	Greedy + interval sorting
“free time,” “busy schedule”	Gap-finding between merged intervals
“employee free time”	Multi-list interval merge

2.2 Hidden Hints

Data given as pairs of `[start, end]` values, even if the problem doesn’t explicitly say “interval,” is a strong tell (e.g., “list of arrival and departure times”).

2.3 Interview Clues

Interviewer draws a timeline on the whiteboard, or the problem’s examples visually resemble a Gantt chart / calendar view.

2.4 Common Trick Words

“Overlap,” “conflict,” “minimum rooms/resources needed,” “can attend all” — all point to interval-based reasoning requiring a sort-then-sweep approach.

2.5 What Interviewers Expect

Correct choice of sort key (by start for merging, sometimes by end for greedy scheduling), correct overlap condition (`next.start <= current.end`, watch for < vs <= boundary semantics depending on whether touching intervals count as overlapping), and clean handling of the “insert a new interval into an already-sorted, already-merged list” variant without re-sorting everything.

2.6 When NOT To Use This Pattern

- Data isn’t naturally range-based (no start/end pairs) — this pattern doesn’t apply.
- You need **point queries** against static ranges repeatedecly (e.g., “is point X covered by any interval, many times”) — consider a different structure (interval tree, or sorted starts + binary search) if updates are frequent.
- Ranges are **multi-dimensional** (2D rectangles) — needs a generalized sweep-line + additional data structure (e.g., segment tree for the second dimension), not just simple 1D sort-and-merge.

SECTION 3 — Decision Framework

```
Is the data a collection of [start, end] ranges?

      Yes
```

```
Do you need to MERGE overlapping ranges or DETECT conflicts?

       Yes → SORT BY START, then sweep and merge adjacent overlaps

        No

Do you need MINIMUM RESOURCES to handle all overlaps simultaneously (e.g., meeting rooms)?

       Yes → SORT START & END SEPARATELY (or use a min-heap of end times) - this is a
              sweep-line / greedy-with-heap hybrid, not simple merging

        No

Do you need to SCHEDULE the MAXIMUM NUMBER of non-overlapping intervals?

       Yes → SORT BY END TIME, apply GREEDY (Pattern #27) selection

        No

Are ranges 2D or need REPEATED point/range queries with updates?

       Yes → Consider Segment Tree / Interval Tree instead (more advanced structure)
```

Why: The sort key changes based on what you're optimizing — "merge" needs start-time sort (to find contiguous overlap groups), "maximum non-overlapping count" needs end-time sort (greedy exchange argument, see Pattern #27), and "minimum concurrent resources" needs to track both boundaries simultaneously (often via a min-heap of active end times).

SECTION 4 — Why This Pattern Works

Mathematical: After sorting by start time, consider intervals `I_1, I_2, ..., I_n` with `start(I_1) <= start(I_2) <= ... <= start(I_n)`. If `I_i` and `I_j` overlap for `i < j`, then since all intervals between them have start times in `[start(I_i), start(I_j)]`, and `I_i`'s end must be `>= start(I_j) >=` every intermediate interval's start, **transitively, `I_i` overlaps with every interval between `i` and `j`** — meaning overlap groups are always **contiguous** in sorted order. This is the key structural fact that permits a single linear sweep to find all merge groups instead of checking all pairs.

Logical: Sorting costs O(n log n); the subsequent linear sweep is O(n) since each interval is examined exactly once. Total: O(n log n), dominated by the sort — versus $O(n^2)$ for pairwise comparison.

Intuitive: Once sorted, "does this new interval overlap with anything I've already grouped" only ever needs to check the **most recently merged interval**, not all previous ones, because of the contiguous-overlap-group property above.

Correctness Proof: *Invariant:* after processing the first `i` sorted intervals, the list of merged intervals so far correctly represents all overlap groups among those `i` intervals. *Base case:* the first interval trivially forms its own group. *Inductive step:* the `(i+1)`th interval either overlaps with the last group's current end (extend it) or doesn't (start a new group) — by the contiguity property, it cannot overlap with any *earlier* group without also overlapping the most recent one (since start times are sorted), so checking only the last group is sufficient. *Termination:* after all `n` intervals are processed, the merged list is complete and correct. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (Merge Overlapping Intervals)

1. Sort intervals by start time.
2. Initialize `merged = [intervals[0]]`.
3. For each subsequent interval: if its start <= the end of the last interval in `merged`, extend that last interval's end to `max(existing end, new end)`; else append it as a new entry.
4. Return `merged`.

5.2 General Template

```
function mergeIntervals(intervals):
    sort intervals by start
    merged = []
    for interval in intervals:
        if merged is empty or merged[last].end < interval.start:
            merged.append(interval)
        else:
            merged[last].end = max(merged[last].end, interval.end)
    return merged
```

5.3 Insert Interval Template

```
function insertInterval(intervals, newInterval):
    result = []
    i = 0
    n = length(intervals)
    # 1. Add all intervals ending before newInterval starts
    while i < n and intervals[i].end < newInterval.start:
        result.append(intervals[i]); i++

    # 2. Merge all overlapping intervals into newInterval
    while i < n and intervals[i].start <= newInterval.end:
        newInterval.start = min(newInterval.start, intervals[i].start)
        newInterval.end = max(newInterval.end, intervals[i].end)
        i++
    result.append(newInterval)

    # 3. Add remaining intervals
    while i < n:
        result.append(intervals[i]); i++

    return result
```

5.4 Minimum Meeting Rooms Template (Heap-Based)

```
function minMeetingRooms(intervals):
    sort intervals by start
    minHeap = new MinHeap()   # tracks end times of ongoing meetings
    for interval in intervals:
        if minHeap not empty and minHeap.peek() <= interval.start:
            minHeap.pop()      # a room freed up
        minHeap.push(interval.end)
    return minHeap.size()      # max concurrent rooms needed
```

5.5 Interview Thinking Process

1. "This is interval data — I'll sort by start time first, since overlap groups are contiguous only after sorting."
2. "I'll maintain the last merged interval and extend it, or start a new group, based on a single comparison per interval."
3. "For 'insert a new interval,' I can avoid re-sorting by processing in three phases: before, overlapping, after."
4. "For 'minimum rooms needed,' I need to track concurrently active intervals — a min-heap of end times, or separately sorted start/end arrays with a two-pointer sweep, both work in O(n log n)."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	O(n log n) (dominated by sort)	O(n) for output/heap	Sort is O(n log n); the merge sweep itself is O(n)
Average Case	O(n log n)	O(n)	Same regardless of overlap density
Best Case	O(n log n) (sort is unavoidable unless pre-sorted)	O(n)	Even fully-overlapping or fully-disjoint inputs need the sort
Amortized	O(n log n) total, O(1) amortized per interval after sorting	O(n)	The sweep itself is linear; sort dominates

If already sorted: the merge sweep alone is O(n), and the pattern's true "core" cost is linear — always check if pre-sorting can be assumed or is a one-time cost across multiple queries.

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty interval list	`[]`	Return empty result immediately
Single interval	`[[1,5]]`	Trivially "merged" as itself
All intervals overlap into one	`[[1,10],[2,5],[3,8]]`	Merges into a single `[1,10]`
No intervals overlap	`[[1,2],[3,4],[5,6]]`	Result equals input, no merging occurs
Touching intervals (`end == next.start`)	`[[1,3],[3,5]]`	Ambiguous — clarify with interviewer whether touching counts as overlapping (commonly yes: <=, but some problems use <)
Negative interval bounds	`[[-5,-1],[-3,2]]`	Works identically — sort/comparison logic doesn't depend on sign

Edge Case	Example	Handling
Nested intervals	`[[1,10],[2,3]]`	The nested interval must not incorrectly shrink the outer one — always use `max(end, ...)`, never overwrite directly
Insert interval that extends beyond all existing	`newInterval=[20,25]` beyond all others	Correctly appended at the end with no merging needed
Insert interval that spans and swallows several existing ones	`newInterval=[0,100]`	All existing intervals merge into the new one; must handle the "many consumed" case in the while loop, not just one

Common mistakes: using `interval.end` directly instead of `max(merged[last].end, interval.end)` when merging — this silently shrinks the merged range if the current interval is nested but shorter; ambiguity about touching-interval semantics not clarified with the interviewer upfront.

SECTION 8 — Pros & Cons

Advantages: O(n log n) total (versus O(n^2) pairwise checking); simple, provably correct sweep after sorting; generalizes cleanly to insert, gap-finding, and resource-counting variants. **Disadvantages:** Requires a sort step (O(n log n)) even if the underlying merge logic is O(n); doesn't handle 2D/multi-dimensional interval overlap without significant extension. **Trade-offs:** Sort-then-sweep (O(n log n)) vs. maintaining a balanced interval tree for dynamic insert/query (O(log n) per operation but higher implementation complexity) — prefer sort-then-sweep for static/batch problems, interval trees for highly dynamic, query-heavy systems. **Limitations:** 1D by default; multi-resource counting (meeting rooms) requires an additional heap/two-pointer layer beyond simple merging. **Inefficient when:** intervals are inserted/queried one at a time in a live system — repeatedly re-sorting is wasteful; use an interval tree or sorted data structure supporting O(log n) insertion instead.

SECTION 9 — Real World Applications

Domain	Application
Google Calendar / Meta Calendar	Merging overlapping busy blocks to compute free/busy time for scheduling suggestions
Amazon	Merging overlapping delivery time windows for route optimization
Uber/Grab	Detecting overlapping ride/driver availability windows for dispatch matching
Airbnb	Merging overlapping booking date ranges to compute host calendar availability
Banking/Payments	Detecting overlapping authorization holds on an account for fraud/limit checks
Networking	Merging overlapping IP address ranges (CIDR block consolidation) for routing table optimization

Domain	Application
Databases	Merging overlapping index range scans during query optimization
Operating Systems	Merging overlapping virtual memory address ranges during memory management
Video Streaming (Netflix)	Merging overlapping buffered time ranges in adaptive streaming players
Project Management Tools (Atlassian/Jira)	Detecting overlapping sprint date ranges or resource allocation conflicts

---

SECTION 10 — Interview Strategy

How seniors answer: They immediately state "I'll sort by start time, since overlap groups become contiguous after sorting — this is the key structural property that makes a single sweep correct," then clarify touching-interval semantics with the interviewer before coding.

How juniors answer: They often attempt pairwise overlap checking ($O(n^2)$) without recognizing the sorting opportunity, or they merge incorrectly by overwriting the end time instead of taking the max, silently breaking on nested intervals.

Typical follow-ups: "What if you need to insert one new interval into an already-merged, sorted list — can you avoid a full re-sort?" "What's the minimum number of meeting rooms needed?" (heap-based sweep). "What if intervals arrive as a live stream, not a batch?" (discuss interval trees / balanced BSTs for O(log n) insert).

Optimization questions: "Can you do the insert-interval variant in O(n) instead of O(n log n)?" (Yes, since the input is already sorted, no full re-sort is needed — just a linear three-phase scan.)

SECTION 11 — Pattern Variations

Variation	Description	Example
Basic Merge	Merge all overlapping intervals	Merge Intervals
Insert Interval	Insert into an already-sorted, already-merged list	Insert Interval
Conflict Detection	Determine if any intervals overlap at all	Meeting Rooms
Minimum Resource Count	Count max concurrent overlaps (heap-based sweep)	Meeting Rooms II
Maximum Non-Overlapping Selection	Greedy selection by end time	Non-overlapping Intervals
Interval Intersection	Find overlapping regions between two separate interval lists	Interval List Intersections
Free Time / Gap Finding	Find gaps between merged busy intervals	Employee Free Time

---

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Greedy	Merge Intervals often uses a greedy exchange argument (sort by end time) for optimal selection sub-problems	Maximum non-overlapping interval count/selection
Sorting	Merge Intervals is a specific application of sort-then-sweep	General sorting doesn't inherently involve range/overlap semantics
Heap/Priority Queue	Used within Merge Intervals for tracking concurrently active ranges (meeting rooms)	Need to track "currently active" state dynamically during the sweep
Sweep Line (general)	Merge Intervals is the simplest 1D instance of the broader Sweep Line technique used in computational geometry	2D/geometric problems need a generalized sweep line with an auxiliary structure

Comparison Table

Aspect	Merge Intervals	Greedy Interval Scheduling	Heap-Based Room Counting
Sort key	Start time	End time	Start time (with end-time heap)
Goal	Consolidate overlaps	Maximize non-overlapping count	Count max concurrent overlaps
Extra structure	None (simple sweep)	None (greedy count)	Min-heap of end times

---

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Basic conflict detection	Meeting Rooms
Medium	Merge and insert operations	Merge Intervals, Insert Interval, Non-overlapping Intervals
Hard	Multi-list intersection, resource counting	Interval List Intersections, Meeting Rooms II
Very Hard	Gap-finding across multiple schedules, 2D extensions	Employee Free Time, My Calendar III

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Merge Intervals	Medium	Amazon, Meta, Microsoft, Google	Direct sort-then-sweep merge	Foundational mechanics
2	Insert Interval	Medium	Amazon, Meta, Google	Three-phase insert without full re-sort	Efficient insert variant
3	Meeting Rooms	Easy	Amazon, Meta, Microsoft	Basic conflict detection	Overlap detection basics
4	Meeting Rooms II	Medium	Amazon, Meta, Google, Microsoft	Min-heap sweep for concurrent count	Heap + interval hybrid
5	Non-overlapping Intervals	Medium	Amazon, Meta, Google	Greedy selection by end time	Greedy + interval combination
6	Interval List Intersections	Medium	Amazon, Meta, Google	Two-pointer sweep across two sorted lists	Multi-list interval handling
7	Employee Free Time	Hard	Amazon, Google, Uber	Merge across multiple employees' schedules + gap-finding	Advanced multi-list gap detection
8	My Calendar I	Medium	Google, Amazon	Incremental conflict detection (single insert at a time)	Dynamic interval insertion
9	My Calendar II	Medium	Google, Amazon	Double-booking detection via overlap counting	Layered overlap counting
10	My Calendar III	Hard	Google	Triple-booking / max concurrent overlap via sweep line	Advanced sweep line counting
11	Minimum Number of Arrows to Burst Balloons	Medium	Amazon, Google	Greedy interval overlap minimization	Greedy + interval combination

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
12	Car Pooling	Medium	Amazon, Google	Difference array / sweep line on pickup-dropoff ranges	Cross-pattern (Prefix Sum + Intervals)
13	Range Module	Hard	Google	Dynamic interval merge/query structure	Advanced dynamic interval structure
14	Add Bold Tag in String	Medium	Amazon	Merge overlapping index ranges	String-index interval merging
15	Teemo Attacking (Poisoned Duration)	Easy	Amazon	Merge overlapping time-effect ranges	Basic interval merge application
16	Video Stitching	Medium	Google	Greedy interval covering	Greedy + interval covering
17	Merge Sorted Array (contrast)	Easy	Amazon	Contrast: value merge, not interval merge	Pattern-boundary awareness
18	Data Stream as Disjoint Intervals	Hard	Google, Amazon	Dynamic interval insertion/merging	Dynamic structure design
19	Falling Squares	Hard	Google	Interval-based height tracking (coordinate compression + intervals)	Advanced interval + compression
20	Summary Ranges	Easy	Amazon, Google	Merge consecutive numbers into ranges	Basic range consolidation
21	Partition Labels	Medium	Amazon, Meta	Related interval-like partitioning via last-occurrence tracking	Adjacent pattern reinforcement
22	Remove Covered Intervals	Medium	Google, Amazon	Sort + containment detection	Containment-based interval logic
23	Maximum Length of Pair Chain	Medium	Amazon, Google	Greedy interval chaining (sort by end time)	Greedy chaining variant

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
24	Interval List Intersections (variant with weights)	Medium	Google (advanced variant)	Weighted interval intersection	Advanced intersection logic
25	The Skyline Problem	Hard	Google, Amazon, Meta	Sweep line with heap for building height intervals	Advanced sweep line + heap
26	Number of Flowers in Full Bloom	Hard	Google	Binary search + interval counting	Cross-pattern (Binary Search + Intervals)
27	Count Integers in Intervals	Hard	Google	Balanced interval merge structure with counting	Advanced dynamic interval counting
28	Minimum Interval to Include Each Query	Hard	Google, Amazon	Sort + heap for query-interval matching	Advanced heap + interval combination
29	Amount of New Area Painted Each Day	Hard	Google (advanced)	Dynamic interval merge with incremental area tracking	Advanced dynamic interval tracking
30	Describe the Painting	Hard	Google	Coordinate compression + interval overlap counting	Advanced coordinate compression + intervals

---

SECTION 15 — Common Mistakes

1. Overwriting the merged interval's end instead of taking `max(existing end, new end)` — silently breaks on nested intervals. *Fix:* always use `max()`.
2. Forgetting to sort before sweeping, or sorting by the wrong key (end instead of start for merge problems). *Fix:* always sort by start for merge/insert; by end for greedy maximum-selection problems.
3. Ambiguous touching-interval handling (< vs <=) not clarified with the interviewer, leading to off-by-one mismatches against expected output. *Fix:* explicitly ask or state the assumption before coding.
4. Attempting to re-sort the entire list for "Insert Interval" when the input is already sorted — wastes the O(n) opportunity for a linear three-phase scan. *Fix:* recognize and exploit the already-sorted precondition.
5. For "minimum meeting rooms," attempting to merge intervals instead of tracking concurrent overlaps via a heap or separate sorted start/end arrays — merging alone doesn't answer "how

many simultaneously," only "do any overlap." *Fix:* recognize this needs a different technique (heap-based sweep) layered on top of sorting.

Why people fail: the core merge logic is simple, but subtle variants (insert without re-sort, minimum concurrent count, touching-interval semantics) require recognizing that "sort by start, sweep, extend end" is not a universal solution — each variant needs a slightly different lens, and candidates who over-generalize the basic template stumble on these.

SECTION 16 — Optimization Techniques

- **Time:** For "Insert Interval," exploit the already-sorted precondition to avoid a full O(n log n) re-sort — use the three-phase O(n) linear scan instead.
- **Space:** Merge in-place where the language/problem allows (e.g., sorting the input array itself, writing merged results back into a prefix of the same array) to avoid O(n) auxiliary space if not needed.
- **Readability:** Clearly separate the "sort" step from the "sweep/merge" step in code; name the running merged interval something explicit like `lastMerged`.
- **Interview performance:** State the sort-key choice and the overlap condition (<= vs <) explicitly and early — this proactively addresses the most common point of ambiguity.

SECTION 17 — Multiple Language Templates

Java

```java
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> merged = new ArrayList<>();
    for (int[] interval : intervals) {
        if (merged.isEmpty() || merged.get(merged.size()-1)[1] < interval[0]) {
            merged.add(interval);
        } else {
            merged.get(merged.size()-1)[1] = Math.max(merged.get(merged.size()-1)[1], interval[1]);
        }
    }
    return merged.toArray(new int[merged.size()][]);
}
```

JavaScript

```javascript
function merge(intervals) {
    intervals.sort((a, b) => a[0] - b[0]);
    const merged = [];
    for (const interval of intervals) {
        if (merged.length === 0 || merged[merged.length-1][1] < interval[0]) {
            merged.push(interval);
        } else {
            merged[merged.length-1][1] = Math.max(merged[merged.length-1][1], interval[1]);
        }
    }
    return merged;
}
```

PHP

```php
function merge(array $intervals): array {
    usort($intervals, fn($a, $b) => $a[0] <=> $b[0]);
    $merged = [];
    foreach ($intervals as $interval) {
        $last = count($merged) - 1;
        if ($last === -1 || $merged[$last][1] < $interval[0]) {
            $merged[] = $interval;
        } else {
            $merged[$last][1] = max($merged[$last][1], $interval[1]);
        }
    }
    return $merged;
}
```

Python

```python
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = []
    for interval in intervals:
        if not merged or merged[-1][1] < interval[0]:
            merged.append(interval)
        else:
            merged[-1][1] = max(merged[-1][1], interval[1])
    return merged
```

Go

```go
func merge(intervals [][]int) [][]int {
    sort.Slice(intervals, func(i, j int) bool { return intervals[i][0] < intervals[j][0] })
    merged := [][]int{}
    for _, interval := range intervals {
        n := len(merged)
        if n == 0 || merged[n-1][1] < interval[0] {
            merged = append(merged, interval)
        } else if interval[1] > merged[n-1][1] {
            merged[n-1][1] = interval[1]
        }
    }
    return merged
}
```

C++

```cpp
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end());
    vector<vector<int>> merged;
    for (auto& interval : intervals) {
        if (merged.empty() || merged.back()[1] < interval[0]) {
            merged.push_back(interval);
        } else {
```

```
            merged.back()[1] = max(merged.back()[1], interval[1]);
        }
    }
    return merged;
}
```

SECTION 18 — Dry Runs

Small Input

`intervals = [[1,3],[2,6],[8,10],[15,18]]` (already sorted by start)

```
merged=[]
[1,3]: merged empty → append → merged=[[1,3]]
[2,6]: 2 <= 3 → merge → merged=[[1,6]]
[8,10]: 8 > 6 → append → merged=[[1,6],[8,10]]
[15,18]: 15 > 10 → append → merged=[[1,6],[8,10],[15,18]]
Result: [[1,6],[8,10],[15,18]]
```

Large Input (Conceptual)

For 10^5 intervals, sorting costs O(n log n) ≈ 1.7×10^6 comparisons, and the sweep is a single O(n) pass — total well within typical interview/production time budgets, and dramatically better than the $O(n^2)$ ≈ 10^10 pairwise comparison alternative.

Corner Case

`intervals = [[1,4],[4,5]]` (touching boundary): with `<=` semantics, `4 <= 4` → merge into `[1,5]`. With strict `<` semantics, they would NOT merge, remaining `[[1,4],[4,5]]` — this exact ambiguity must be clarified with the interviewer before finalizing.

SECTION 19 — Advanced Concepts

- **Sweep Line generalization:** Merge Intervals is the simplest 1D case of the **Sweep Line** technique used broadly in computational geometry (e.g., The Skyline Problem, finding rectangle union areas) — events (starts as +1, ends as -1) are processed in sorted order while maintaining a running "active count" or "active state."
- **Heap-based concurrent counting:** for "minimum meeting rooms," maintaining a min-heap of active end times lets you reuse a room the moment its meeting ends — the heap's size at any point represents the concurrent room requirement, and its maximum across the sweep is the answer.
- **Two-array sweep (alternative to heap):** separately sort all start times and all end times; walk both with two pointers, incrementing a counter on a start and decrementing on an end that occurs no later — this achieves the same O(n log n) result as the heap approach with potentially simpler code.
- **Coordinate compression:** for problems with very large or continuous ranges (e.g., "Falling Squares," "Describe the Painting"), compress the relevant boundary coordinates into a smaller discrete index space before applying interval logic, to avoid iterating over an enormous or infinite coordinate range.

SECTION 20 — Senior Engineer Insights

Staff engineers recognize that Merge Intervals is a specific instance of the general principle: **sorting by one dimension to make a global relationship (overlap) into a local, adjacent one** — the same insight underlies sweep-line algorithms in computational geometry, event-driven simulation systems, and even database query optimizers reasoning about overlapping index ranges. Interviewers watch for whether a candidate recognizes when simple merging is insufficient (e.g., needing a heap for "concurrent count" rather than just "any overlap") and whether they can extend the technique to multi-list, multi-dimensional, or streaming variants without needing to be walked through it — these extensions are what separate Staff-level pattern fluency from rote memorization of the basic merge template.

SECTION 21 — Cheat Sheet

```
PATTERN: Merge Intervals
RECOGNIZE: "intervals," "meetings," "ranges," "schedule," "overlap," "merge," "conflict"
TEMPLATE (merge):
    sort by start
    merged = [intervals[0]]
    for each interval:
        if interval.start <= merged[-1].end: merged[-1].end = max(merged[-1].end, interval.end)
        else: merged.append(interval)
COMPLEXITY: O(n log n) time (sort-dominated), O(n) space
KEY PROOF: after sorting by start, overlap groups are always contiguous - only need to check the last grou
WATCH FOR: touching-interval semantics (<= vs <), max() not overwrite for nested intervals, correct sort k
DOESN'T APPLY WHEN: non-range data, 2D overlap without sweep-line extension, highly dynamic single-insert-
```

SECTION 22 — Revision Notes (5-Minute Review)

- Sort by start time → overlap groups become contiguous → single O(n) sweep suffices.
- Merge condition: `interval.start <= lastMerged.end` → extend with `max()`, never overwrite.
- "Insert Interval" into already-sorted data: three-phase linear scan (before/overlapping/after), no re-sort needed.
- "Minimum meeting rooms" needs concurrent-count tracking (min-heap of end times or two-pointer start/end sweep), not simple merging.
- "Maximum non-overlapping selection" is a Greedy problem sorted by end time (Pattern #27), a different sort key than merging.
- Clarify touching-interval semantics with the interviewer explicitly.

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic merge and conflict detection	Merge Intervals (56), Meeting Rooms (252)
Intermediate	Insert and greedy selection	Insert Interval (57), Non-overlapping Intervals (435), Minimum Number of Arrows to Burst Balloons (452)

Stage	Focus	Recommended LeetCode Problems
Advanced	Concurrent counting, multi-list intersection	Meeting Rooms II (253), Interval List Intersections (986), My Calendar II (731)
Expert	Sweep line, gap-finding, advanced counting	Employee Free Time (759), The Skyline Problem (218), My Calendar III (732)

SECTION 24 — Memory Tricks

- **Mnemonic:** "**S**ort, **S**weep, **E**xtend" (SSE) — Sort by start, Sweep left to right, Extend the current merged end.
- **Visualization:** A **hotel booking calendar** — overlapping reservation blocks visually merge into one continuous "occupied" bar.
- **Recognition shortcut:** "[start, end]" pairs + "overlap/merge/schedule/conflict" → sort by start, sweep, extend.

SECTION 25 — Final Summary

Merge Intervals exploits a simple but powerful structural fact: **once ranges are sorted by start time, overlap groups become contiguous**, collapsing an apparent $O(n^2)$ pairwise comparison problem into an O(n log n) sort-then-sweep. The single most important thing to remember forever: **always sort first — by start for merging/inserting, by end for greedy maximum-selection — and remember that "any overlap" (simple merge) and "how many overlap simultaneously" (concurrent count via heap) are different questions requiring different techniques layered on top of the same sorted foundation.**